Multi-Agent 개념

1-1. 왜 여러 Agent인가
1-2. 협업 구조의 종류
1-3. 언제 Multi-Agent를 쓰지 말아야 하는가
1-4. 쉬어가기: Multi-Agent 회의론도 같이 읽기

1-1. 왜 여러 Agent인가

> 한 Agent에 도구를 몰아줄 때 생기는 도구 선택 혼란을 직접 재현해봅니다.

5일차까지 만든 구조는 단일 Agent가 도구 4-5개를 들고 있는 형태였습니다. 도구가 그 정도일 때는 잘 동작합니다. 문제는 도구가 8개, 10개로 늘어날 때 시작됩니다.

늘어나는 부담	나타나는 증상
시스템 프롬프트 폭발	이 도구는 이럴 때, 저 도구는 저럴 때라는 안내가 30줄 이상으로 길어집니다
도구 선택 정확도 하락	의미가 겹치는 도구가 많으면 모델이 잘못 고르는 빈도가 눈에 띄게 늘어납니다
컨텍스트 오염	문서 검색 결과가 데이터 조회 작업 중간에 끼어들어 대화가 산만해집니다
디버깅 어려움	왜 이 도구를 골랐는지 추적하려 해도 모든 결정이 한 노드에 뭉쳐 있습니다
모델 선택 경직	조회는 저렴한 모델로 충분한데 분석에는 강한 모델이 필요합니다. 한 모델로는 둘 다 못 맞춥니다

여기서 Multi-Agent의 핵심을 정확히 잡고 가야 합니다. AI 여러 명이 토론하면 더 똑똑해진다가 핵심이 아닙니다. 핵심은 복잡한 일을 역할별로 나눠 각 Agent가 더 작은 문제만 보게 만드는 것입니다.

따라하기

1. 의미가 겹치는 도구 8개를 한 Agent에 몰아주고 애매한 질문을 반복해 던져 봅니다. 아래 코드를 day6_practice 폴더에 `single_agent_problem.py` 로 저장해 실행합니다.

```python
# single_agent_problem.py - 단일 Agent의 도구 혼동 재현
from dotenv import load_dotenv
from langchain_aws import ChatBedrockConverse
from langchain_core.tools import tool
from langchain_core.messages import HumanMessage
from langgraph.prebuilt import create_react_agent

load_dotenv()

# 교육용 더미 데이터를 돌려주는 도구 8개입니다
@tool
def retrieve_docs(query: str) -> str:
    """사내 정책 문서를 검색한다."""
    return f"[문서] {query} 관련: 연차 규정, 재택근무 규정, 출장 규정"

@tool
def search_budget_policy(query: str) -> str:
    """예산 관련 정책 문서를 검색한다."""
    return "[문서] 예산 정책: 분기 예산은 전 분기 실적 기준으로 편성"

@tool
def search_employees(team: str) -> str:
    """팀별 임직원 정보를 사내 DB에서 조회한다."""
    return f"[DB] {team} 임직원: 김하늘, 이서준, 정하은"

@tool
def search_expenses(team: str) -> str:
    """팀별 비용 집행 내역을 사내 DB에서 조회한다."""
    return f"[DB] {team} 비용: 물류 최적화 1억 2천(초과), 데이터 파이프라인 2천 8백"

@tool
def search_projects(team: str) -> str:
    """팀별 프로젝트 목록을 사내 DB에서 조회한다."""
    return f"[DB] {team} 프로젝트: 물류 최적화, 데이터 파이프라인"

@tool
def search_assets(team: str) -> str:
    """팀별 자산 목록을 사내 DB에서 조회한다."""
    return f"[DB] {team} 자산: 노트북 2대, 모니터 2대, GPU 서버 1대"

@tool
def write_report(content: str) -> str:
    """마크다운 보고서를 작성한다."""
    return f"# 보고서\n\n{content}"

@tool
def send_message(channel: str, msg: str) -> str:
```

```
    """사내 메신저로 메시지를 발송한다."""
    return f"[메신저] {channel}: {msg}"

ALL_TOOLS = [retrieve_docs, search_budget_policy, search_employees,
             search_expenses, search_projects, search_assets,
             write_report, send_message]

llm = ChatBedrockConverse(
    model="us.anthropic.claude-sonnet-4-5-20250929-v1:0",
    region_name="us-east-1",
)
fat_agent = create_react_agent(
    llm,
    ALL_TOOLS,
    prompt="너는 삼성SDS 만능 어시스턴트다. 알맞은 도구를 골라 답하라.",
)

question = "클라우드운영팀 예산 초과 항목 알려줘"
for i in range(5):  # 같은 질문을 5회 반복해 선택이 흔들리는지 봅니다
    result = fat_agent.invoke({"messages": [HumanMessage(question)]})
    tools_used = [
        tc["name"]
        for m in result["messages"]
        if getattr(m, "tool_calls", None)
        for tc in m.tool_calls
    ]
    print(f"{i + 1}회 호출 도구: {tools_used}")
```

2. 출력을 보고 몇 회에서 문서 검색 도구와 데이터 조회 도구를 혼동했는지 기록합니다. 도구 목록을 4개로 줄여 다시 실행하면 혼동이 줄어드는 것도 함께 비교합니다.

막힐 때

- 혼동이 재현되지 않습니다: 최신 모델은 도구 8개 정도는 잘 처리하기도 합니다. 도구 이름을 더 비슷하게 만들거나 docstring을 짧고 모호하게 줄이면 재현됩니다. 핵심은 도구가 늘어나고 설명이 겹칠수록 정확도가 떨어진다는 경향을 확인하는 것입니다.
- `create_react_agent` import 오류: 설치된 버전에 따라 5일차에서 쓴 `from langchain.agents import create_agent` 를 그대로 써도 됩니다. 인자 구조는 거의 같고 프롬프트 인자 이름만 `prompt` 와 `system_prompt` 로 다릅니다.

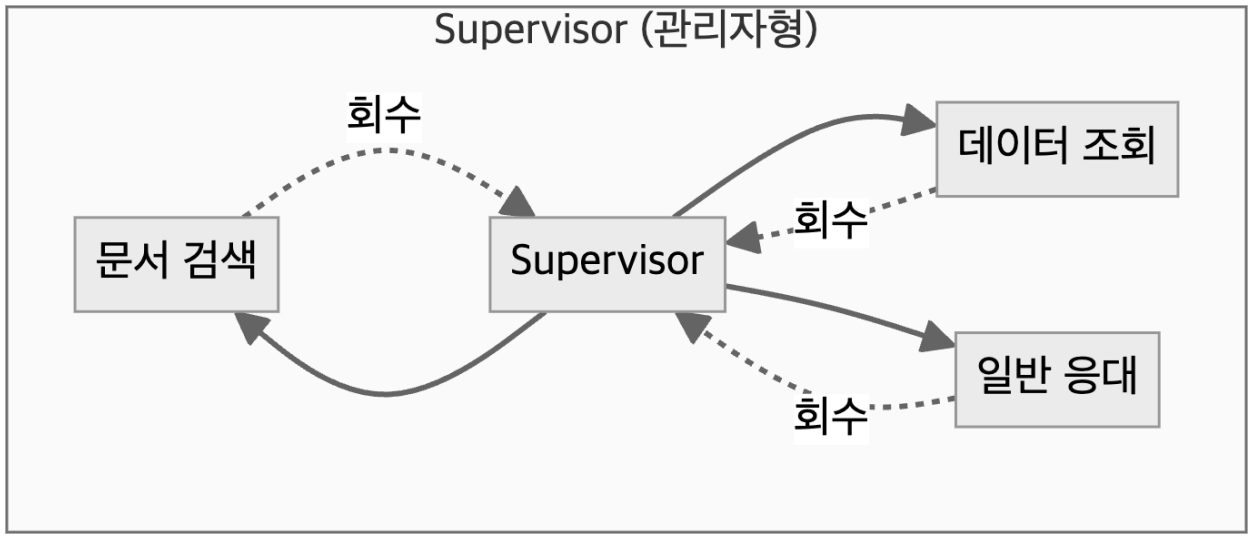
확인: 5회 중 한 번이라도 `search_budget_policy` 처럼 문서 검색 도구가 불려 나오면 혼동을 재현한 것이고 도구를 줄였을 때 그 빈도가 낮아지면 성공입니다.

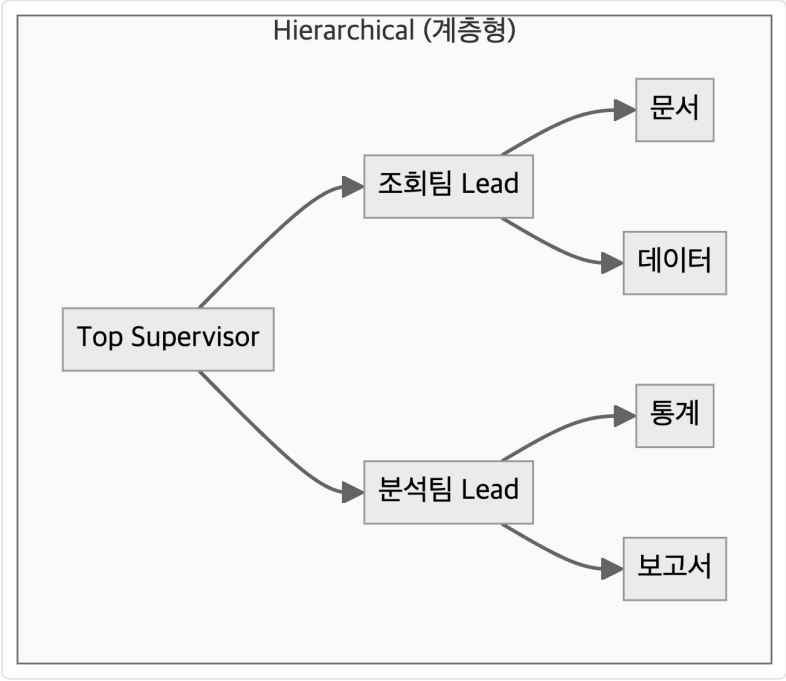
> 도구가 늘어날수록 단일 Agent의 선택 정확도가 떨어진다는 걸 직접 확인했습니다.

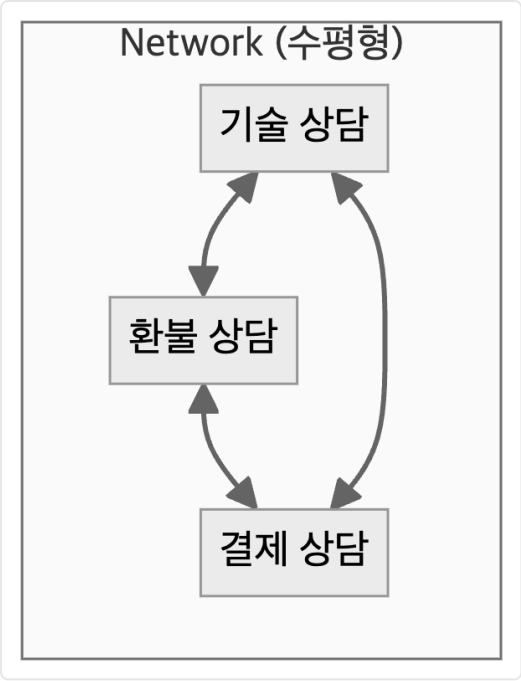
1-2. 협업 구조의 종류

> Supervisor, Hierarchical, Network 세 협업 구조를 구분해 이해해봅니다.

Multi-Agent 협업 구조는 크게 세 가지로 나뉩니다. LangGraph는 세 가지를 모두 표현할 수 있습니다.







패턴	흐름 제어	특징	적합한 경우
Supervisor	중앙 1명	단순하고 추적이 쉽습니다	도메인이 명확히 나뉠 때
Hierarchical	중앙 계층	큰 시스템을 부서 단위로 나눕니다	하위 Agent가 5개를 넘고 팀 경계가 보일 때
Network	분산	유연하지만 추적이 어렵습니다	협업 흐름을 미리 정하기 어려울 때

오늘의 주인공은 Supervisor입니다. 한 명의 관리자 Agent가 요청을 해석해 하위 Agent에게 배분하고 각 Agent는 자기 영역만 처리해 결과를 돌려줍니다. 그러면 관리자가 결과를 모아 최종 답을 만듭니다.

가장 자주 쓰이고 추적이 쉬워서 처음 만드는 업무 시스템에 가장 안전한 선택입니다.

> 세 협업 구조의 차이와 Supervisor부터 시작하는 이유를 이해했습니다.

1-3. 언제 Multi-Agent를 쓰지 말아야 하는가

> 단일 Agent로 충분한 경우를 구분하는 판단 기준을 세워봅니다.

Multi-Agent는 공짜가 아닙니다. Agent가 늘어나면 모델 호출도 그만큼 늘어납니다. 흐름 추적이 어려워지고 한 Agent의 실패가 전체로 번지기도 합니다.

그래서 쓰지 말아야 할 때를 판단하는 기준이 쓰는 법만큼 중요합니다.

단일 Agent로 충분한 경우입니다.

1. 도구가 5개 이하이고 서로 잘 구분될 때입니다. 도구 설명이 겹치지 않으면 단일 Agent도 충분히 정확하게 고릅니다.
2. 작업들이 컨텍스트를 전부 공유해야 할 때입니다. 코드 수정 작업처럼 파일 전체 맥락을 한 Agent가 계속 들고 있어야 하는 경우입니다. 이때는 쪼갤 때 드는 맥락 전달 비용이 이득보다 큽니다.
3. 단계가 항상 고정되어 있을 때입니다. A 다음 무조건 B, 그다음 무조건 C라면 라우팅(요청을 어느 Agent에게 보낼지 정하는 일) 결정 자체가 필요 없으므로 1일차에 배운 순차 체인이 정답입니다.
4. 지연 시간과 비용에 민감할 때입니다. Agent 5개면 모델 호출이 최소 5배입니다.

Multi-Agent가 맞는 신호입니다.

1. 시스템 프롬프트가 도구 안내로 30줄을 넘기 시작했습니다.
2. 의미가 비슷한 도구 사이의 오선택이 실행 기록에서 반복 관찰됩니다.
3. 도메인별로 다른 모델을 쓰고 싶습니다.
4. 팀별로 Agent를 독립적으로 개발하고 평가해야 합니다.
5. 여러 출처를 병렬로 뒤져 종합해야 합니다.

판단의 단위는 도구 개수가 아니라 도메인입니다. 도구 8개니까 Agent 8개는 과도한 분해입니다. 오늘은 사내 데이터 조회, 문서 검색, 일반 응대라는 3개 도메인으로 나눕니다.

따라하기

1. 아래 네 상황에서 단일 Agent, Multi-Agent, 체인 중 무엇을 고를지 먼저 스스로 답한 뒤 답을 확인합니다.

```
1) 도구 3개(계산기, 날씨, 검색)를 쓰는 개인 비서 챗봇
2) 문서 검색 4개 + 데이터 조회 5개 + 발송 3개 도구를 쓰는 사내 어시스턴트
3) 고정 순서로 문서 로드에서 요약, 번역까지 수행하는 파이프라인
4) 여러 출처를 동시에 뒤져 시장 조사 보고서를 만드는 시스템
```

2. 자신의 실제 업무 중 하나를 골라 위 기준으로 판정하고 그 근거를 한 줄로 적어 둡니다. 이 판단이 Mini PJT 주제 선정의 출발점이 됩니다.

▼ 판단 연습 답

1번은 단일 Agent입니다. 도구가 적고 서로 명확히 구분됩니다. 2번은 Multi-Agent입니다. 도메인이 3개로 자연스럽게 나뉩니다. 3번은 체인입니다. 라우팅 결정이 아예 필요 없습니다. 4번은 Multi-Agent입니다. 병렬 조사 후 종합이 필요합니다.

막힐 때

- 우리 프로젝트는 애매합니다: 애매하면 단일 Agent로 시작해 위의 신호가 실행 기록에서 관찰될 때 분리합니다. 처음부터 쪼개는 것보다 필요해졌을 때 쪼개는 쪽이 항상 쉽습니다.
  Supervisor 구조는 기존 단일 Agent를 하위 Agent 하나로 넣는 것부터 시작할 수 있어 옮기는 비용도 크지 않습니다.

확인: 네 상황의 판정이 답과 일치하고 자신의 업무를 판정한 근거를 한 줄로 적었으면 성공입니다.

> 도구 개수가 아니라 도메인이 분리 기준이라는 원칙을 익혔습니다.

1-4. 쉬어가기: Multi-Agent 회의론도 같이 읽기

> Multi-Agent를 놓고 갈린 두 주장을 보고 오늘 만들 구조가 어느 쪽인지 판단해봅니다.

2025년에 Multi-Agent를 놓고 정반대 주장이 거의 동시에 나왔습니다. 양쪽 다 알아 두면 설계할 때 덜 헤맵니다.

- 회의론: Agent를 나누면 컨텍스트가 쪼개진다. 서로 무슨 결정을 했는지 모른 채 각자 부분 답을 만들고, 그걸 합치면 앞뒤가 안 맞는 결과가 나온다. 차라리 하나의 흐름에 컨텍스트를 모으는 게 낫다.
- 지지론: 조사처럼 넓게 훑어야 하는 작업에서는 여러 Agent가 병렬로 다른 영역을 파고들 때 성능이 뚜렷하게 올랐다. 토큰을 더 쓰고 품질을 산 셈이다.

두 주장은 사실 싸우고 있지 않습니다. 갈림길은 하나입니다.
작업이 독립적으로 쪼개지면 나누는 게 이득이고, 중간 결정이 서로 얽혀 있으면 나누는 순간 손해입니다.

오늘 만드는 Supervisor가 비교적 안전한 것도 그래서입니다. 하위 Agent가 전체 messages를 공유하니 회의론이 지적한 컨텍스트 분절이 덜 일어납니다.
반대로 2번째 세션 2-4에서 볼 agents-as-tools는 질문 문자열만 넘기므로 이 위험이 더 큽니다.

1-3에서 정리한 Multi-Agent를 쓰지 말아야 하는 경우를 다시 읽어 보면 이 논쟁의 요약이라는 걸 알 수 있습니다.

> 쪼갤 수 있는 작업이면 나누고 결정이 얽히면 합친다는 기준을 잡았습니다.